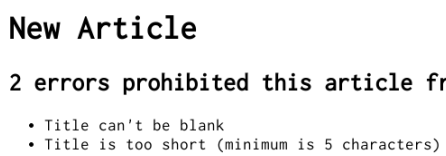
```
<%= link_to 'Back', articles_path %>
```

A few things are going on. We check if there are any errors with `@article.errors.any?`, and in that case we show a list of all errors with `@article.errors.full_messages`.
`pluralize` is a rails helper that takes a number and a string as its arguments. If the number is greater than one, the string will be automatically pluralized.
The reason why we added `@article = Article.new` in the `ArticlesController` is that otherwise `@article` would be `nil` in our view, and calling `@article.errors.any?` would throw an error.
Rails automatically wraps fields that contain an error with a div with class `field_with_errors`. You can define a css rule to make them standout.

Now you'll get a nice error message when saving an article without title when you attempt to do just that on the new article form http://localhost:3000/articles/new:



5.11 Updating Articles

We've covered the "CR" part of CRUD. Now let's focus on the "U" part, updating articles.

The first step we'll take is adding an `edit` action to the `ArticlesController`, generally between the `new` and `create` actions, as shown:

```
def new
  @article = Article.new
end

def edit
  @article = Article.find(params[:id])
end

def create
  @article = Article.new(article_params)

  if @article.save
    redirect_to @article
  else
    render 'new'
  end
end
```

The view will contain a form similar to the one we used when creating new articles. Create a file called `app/views/articles/edit.html.erb` and make it look as follows:

```
<h1>Editing article</h1>

<%= form_for :article, url: article_path(@article), method: :patch do
|f| %>

  <% if @article.errors.any? %>
    <div id="error_explanation">
      <h2>
        <%= pluralize(@article.errors.count, "error") %> prohibited
        this article from being saved:
      </h2>
      <ul>
        <% @article.errors.full_messages.each do |msg| %>
          <li><%= msg %></li>
        <% end %>
```